Group Assignment Sheet

Subject Code: ICT 3128

Subject Name: Wireless Communication and Computing

Date: 25/10/2026

Section: A

Group Number: 5

Question Number: 5

Project Title: MFSK Telemetry Transceiver under AWGN and Narrowband Interference

Team Lead: Aryan Malik

Group Members and Contributions

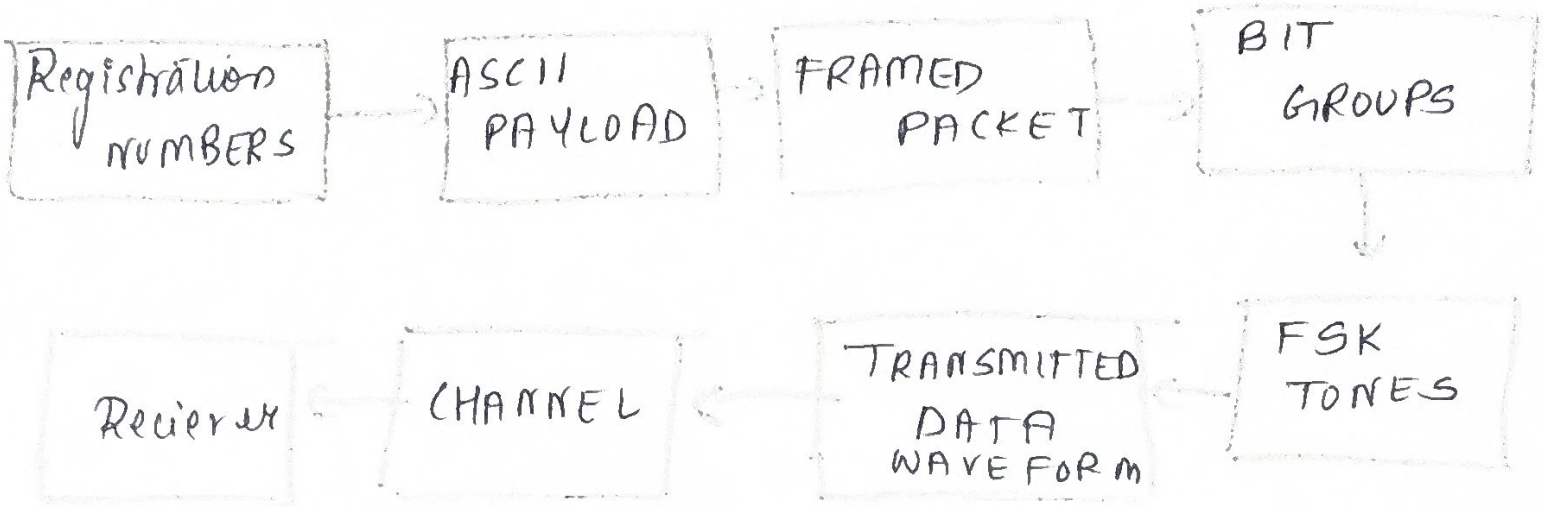
Name	Registration Number	Sub-question completed	Contribution
B RASHMI KINI	240953568	5A	Transmitter design, MATLAB implementation, calculations and plots
ARYAN MALIK	240953668	5B	AWGN and narrowband interference channel model
AYATI RAUTHAN	240953138	5C	Receiver design, frame recovery, CRC validation and BER/PER
AKSHAT MANNUR	240953634	5D	Results comparison and final engineering decision

B Rashmi Kini
24095 3568
Rashmi Kini

A.1:

Aim: Registration numbers and settings

Transmitter Design And Implimentation




```
%% 1. Input data and settings
numbers = {'240953668','240953568','240953138','240953634'};
payloadText = strjoin(numbers, '|'); % | separates the four numbers
payloadBytes = uint8(payloadText);  % ASCII code for each character
Rs = 300;        % symbols per second
Ts = 1/Rs;       % seconds per symbol
spacing = 1/Ts; % tone spacing: 300 Hz, orthogonal over Ts
fc = 3000;       % center frequency (change this if required)
Fs = 24000;      % samples per second
Ns = Fs/Rs;      % samples per symbol: 80
```

The aim is to construct a telemetry transmitter using our 4 registration numbers and produce both 4-FSK and 8 FSK signals. The four registration numbers are 24095368, 24095368, 24095138 and 24095634. The code stores them as text. It joins them with the character |, producing 24095368|24095358|24095138|24095634. The three separator allow the four numbers to be separated against at the reciever.

The code converts each character to an ASCII byte. For example, the first character '2' has ASCII value 50, represented by 00110010 in binary. It does not convert the

B Rashmi Kini
Karthik

entire a digit registration number into one binary number. There are $4 \times 9 = 36$ digit characters and three 1 characters Therefore the payload has 39 characters = 39 bytes = 312 bits.

The remaining numbers in this code is transmitter settings. $R_s = 300$ symbols/s means the transmitter sends 300 FSK symbols each second. Hence one symbol lasts $T_s = 1/300 = 3.333$ ms $f_c = 3000$ Hz is the choosen centre of the tone allocation; it is not taken from a registration numbers. $F_s = 24000$ samples/s means MATLAB calculates 24000 ~~samples/s~~ waveform values per second. Consequently, it calculates $F_s/R_s = 24000/300 = 80$ waveform values for each symbol. The tone spacing is choosen as $1/T_s = 300$ Hz

A.2: Packet, CRC and Packet output

The transmitter does not send only the 312 payload bits. It first makes a framed packet. The first two bytes AS 5A in hexadecimal, are the preamble. They form a known starting pattern that the receiver can search for The next byte stores the number 39, telling the receiver that 39 payload bytes follow. The payload contains our four registration numbers and separators. The final

two bytes contains the CRC.

B Rashmi Kini
240953568
Rashmi

```
Payload: 240953668|240953568|240953138|240953634
Payload = 39 bytes; frame = 352 bits; CRC = A327 (hex)
```

```
%% 2. Packet = preamble + length + payload + CRC
preamble = uint8([165 90]); % hexadecimal A5 5A
lengthByte = uint8(numel(payloadBytes));
checkBytes = [lengthByte payloadBytes];

% CRC-16/CCITT-FALSE over the length and payload bytes.
% Initial value FFFF; polynomial 1021; no final XOR.
crc = uint16(65535);
for n = 1:numel(checkBytes)
    crc = bitxor(crc, bitshift(uint16(checkBytes(n)), 8));
    for b = 1:8
        if bitget(crc, 16) == 1
            crc = bitxor(bitshift(crc, 1), uint16(hex2dec('1021')));
        else
            crc = bitshift(crc, 1);
        end
    end
end
```

```
crcHigh = uint8(bitshift(crc, -8));
crcLow = uint8(bitand(crc, uint16(255)));
frameBytes = [preamble checkBytes crcHigh crcLow];

% Change every packet byte into 8 bits, most significant bit first.
frameBits = zeros(1, 8*numel(frameBytes));
position = 1;
for n = 1:numel(frameBytes)
    for bitNumber = 8:-1:1
        frameBits(position) = bitget(frameBytes(n), bitNumber);
        position = position + 1;
    end
end
fprintf('Payload: %s\n', payloadText);
fprintf('Payload = %d bytes; frame = %d bits; CRC = %s (hex)\n\n', ...
    numel(payloadBytes), numel(frameBits), dec2hex(crc,4));
```

The code calculates CRC-16/CCITT-FALSE using initial value FFFF and polynomial 1021 in hexadecimal. It processes the length and payload bytes. In the loop, each byte is combined with the CRC register, and its eight bits are processed by shifting & applying the polynomial when required. For our packet, MATLAB obtains A327. At the receiver, a new CRC is calculated and can be compared with this value to detect a changed bit.

Next frombytes, puts the preamble, length, payload and CRC in the order in which they are sent. The nested loop converts each byte into eight bits starting with its most significant bit. For example, A5 becomes 10100101. Therefore the first bits in the packet belong to the preamble, the registration-number bits appear after the preamble and length byte.

$$\underbrace{2}_{\text{preamble}} + \underbrace{1}_{\text{length}} + \underbrace{39}_{\text{payload}} + \underbrace{2}_{\text{CRC}} = 44 \text{ bytes} = 352 \text{ bits}$$

A-3:
How bits select frequencies

```
%% 3. Make the 4-FSK and 8-FSK waveforms from the same packet
Mvalues = [4 8];
signals = cell(1,2);  % signals{1}: 4-FSK; signals{2}: 8-FSK
tones = cell(1,2);
symbolTones = cell(1,2);
paddingBits = zeros(1,2);

for q = 1:2
    M = Mvalues(q);
    k = log2(M);                          % 4-FSK: 2 bits; 8-FSK: 3 bits
    paddingBits(q) = mod(-numel(frameBits), k);
    bits = [frameBits zeros(1, paddingBits(q))];
    numberOfSymbols = numel(bits)/k;
    tones{q} = fc + ((0:M-1) - (M-1)/2)*spacing;
    symbolTones{q} = zeros(1, numberOfSymbols);
```

```
4-FSK tones (Hz): 2550  2850  3150  3450
Rs = 300 symbols/s; gross bit rate = 600 bits/s
Tone span = 900 Hz; approximate bandwidth = 1500 Hz
Symbols = 176; padding = 0 bits; time = 0.5867 s

8-FSK tones (Hz): 1950  2250  2550  2850  3150  3450  3750  4050
Rs = 300 symbols/s; gross bit rate = 900 bits/s
Tone span = 2100 Hz; approximate bandwidth = 2700 Hz
Symbols = 118; padding = 2 bits; time = 0.3933 s
```

B Rashmi Kini

FSK sends information by shifting a wave Rashmi frequency for each symbol. A frequency such as 3150Hz means the wave makes 3150 complete cycles in one second. The frequency is not a registration number, it is the physical-wave choice that represents a group of packet bits. The same 952 packet bits are used in both FSK modes.

With 4-FSK $M=4$, so $\log_2 4 = 2$ bits choose one of 4 tones. With 8-FSK, $M=8$, so $\log_2 8 = 3$ bits choose one of eight tones. The code converts a bit group from binary to a decimal toneIndex. Since MATLAB position starts at 1 while our tone index start at 0, toneIndex+1 selects the correct entry from the tone array.

Tones are placed around the shifted center $f_c = 3000Hz$. Their spacing is $\Delta f = 1/T_s = 300Hz$. This means each neighbouring frequency differs by 300Hz & $\Delta f T_s = 1$, the choosen orthogonal spacing over one symbol interval

$$f_i = f_c + \left(i - \frac{(M-1)}{2}\right)\Delta f.$$

4-FSK

Bits	Index i	Frequency Calculation
00	0	$3000 + (0-1.5)300 = 2550Hz$
01	1	$3000 + (1-1.5)300 = 2850Hz$
10	2	$3000 + (2-1.5)300 = 3150Hz$
11	3	$3000 + (3-1.5)300 = 3450Hz$

Thus if the next 2 bits are 10, then index is 2 and the transmitter generates a 2150 Hz wave for one 2.233 ms symbol. For 8-FSK the bit groups 000 through 111 select, in order, 1950, 2250, 2550, 2850, 3150, 3450, 3750, 4050 Hz.

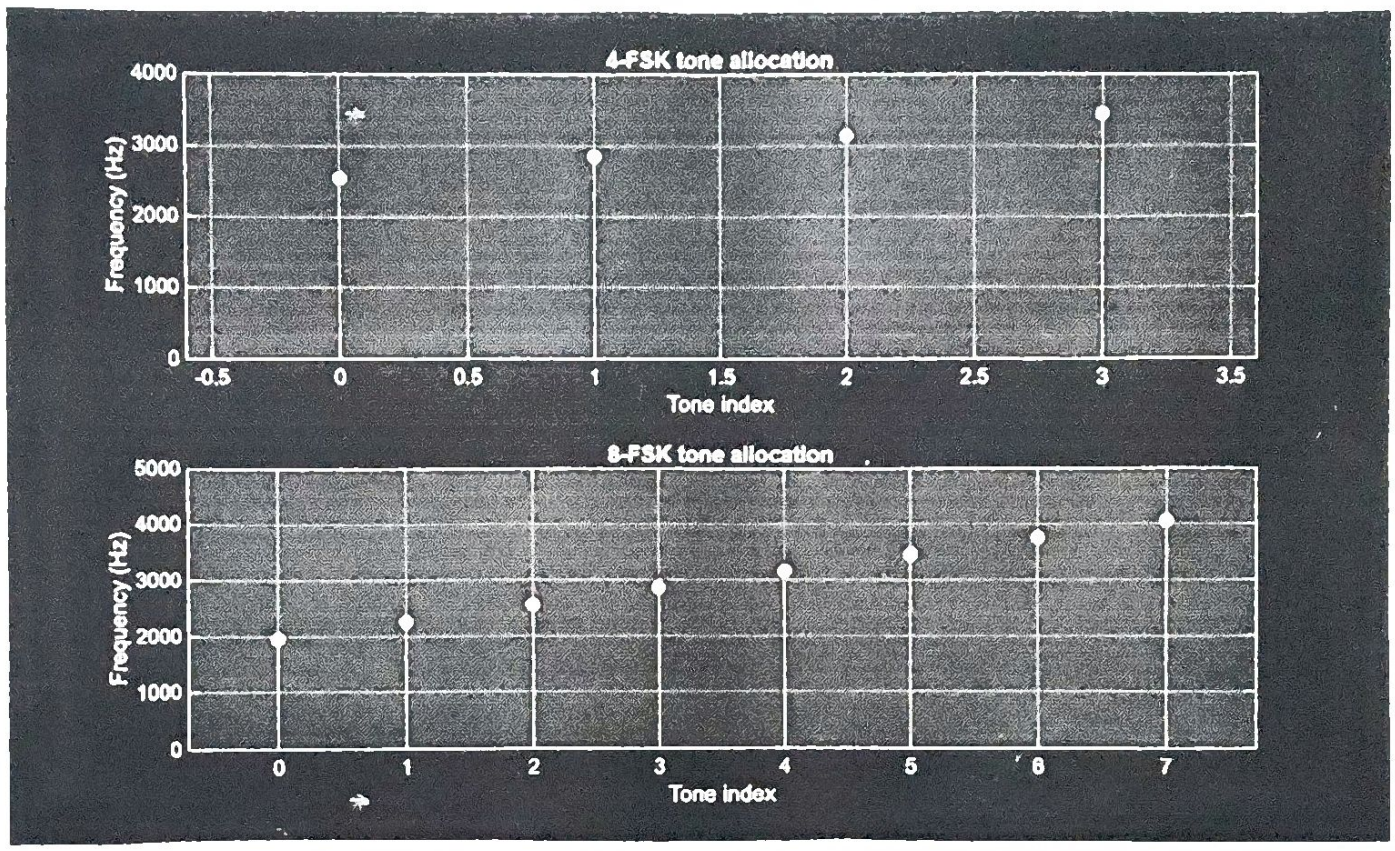


Figure 1

Figure 1 plots tone index on the x axis and its frequency in hertz on the y-axis. The upper graph gives the four choices for 4FSK and lower gives the eight choices for 8FSK It tells us which wave frequency each bit group will select. It does not show how the wave changes with time. The 8-FSK frequencies cover a wider range because eight choices, rather than four, are placed 300 Hz apart.

B Roshmi Kini

HOW MATLAB CREATES THE WAVE

A.K

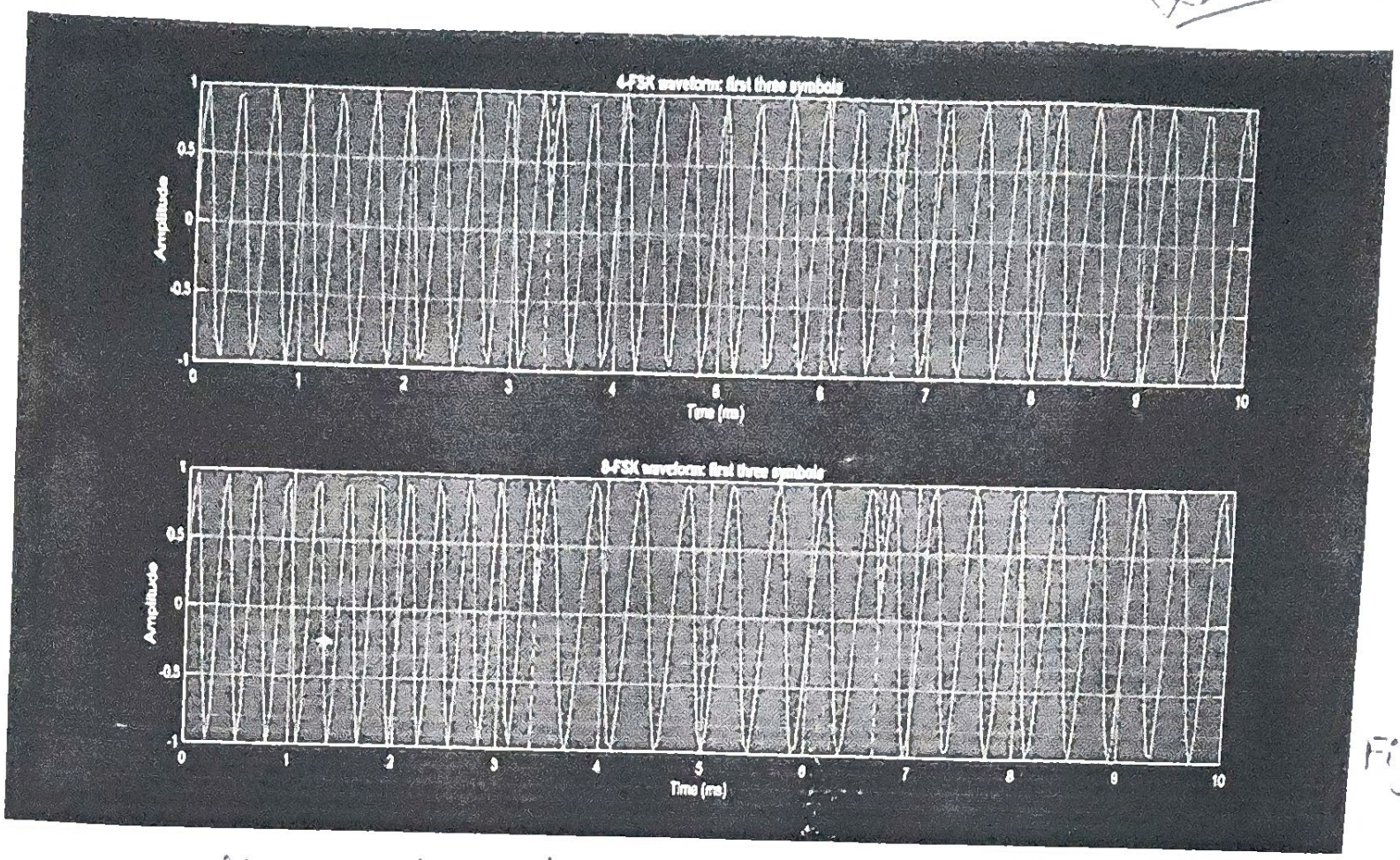


Figure 2.

Each bit group selects 1 frequency. MATLAB then calculates 80 amplitude values for that symbol using $s(t) = \sin(2\pi f t)$ with samples $1/24000$ second apart. For a 3150 Hz tone, the first value is $\sin(0) = 0$, and the next is $\sin(2\pi \times 3150/24000)$.

So 0.13 ms plus 80 these symbols in the correct part of the complete waveform.

Figure 2 shows the first 240 samples $240/24000 = 10$ ms of three symbols. The top graph is 4-FSK and the bottom is 8FSK. Dashed lines mark the symbol ~~duration~~ boundaries. The first preamble groups select 3150, 3150, 2850 Hz (10) (10) (01) & 8-FSK - 3450, 2250, ~~2250~~, 2550 Hz (101) (001) (010). The payload carries data in the full waveform.

A:5:

Pointed Results & Calculations

Dashwin

Both modes send 300 symbols/s. Each 4-FSK symbol carries 2 bits, giving $300 \times 2 = 600$ bits/s. Each 8-FSK symbol carries 3 bits giving $300 \times 3 = 900$ bit/s. These are gross bit rates. ($R_b = R_s \log_2 M$)
The code estimates bandwidth as the span between the outer tone centres plus $2/T_s = 600$ Hz for spectral spread.

	4-FSK	8-FSK
Tone frequencies	1550 - 3450 Hz (4 tones)	1950 - 4050 Hz (8 tones)
Tone centre span	900 Hz	2100 Hz
Approximate Bandwidth	1500 Hz	2700 Hz
Packet symbols	352/2 = 176	(352+2)/3 = 118
Padding	0 bits	2 bits
Packet duration	176/300 = 0.5867 s	118/300 = 0.3933 s

A.6: PSD, saved output

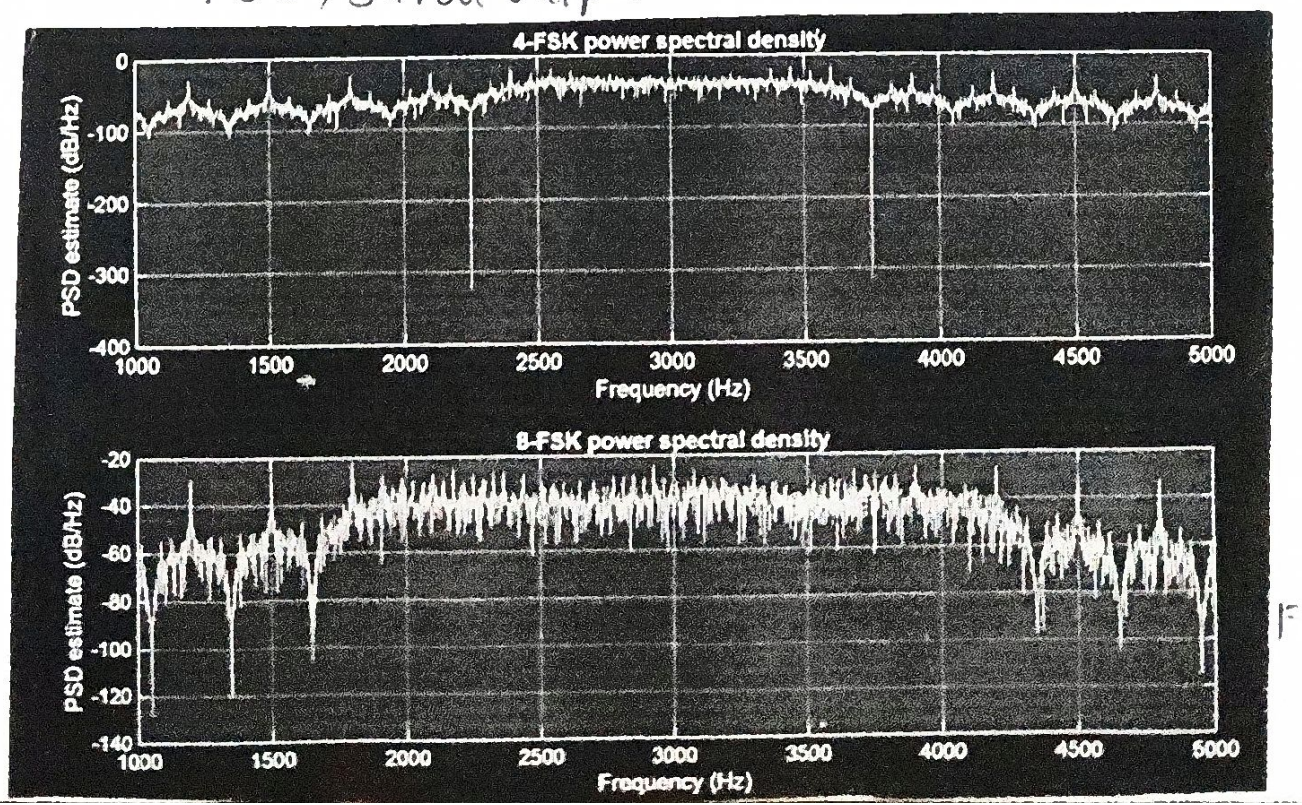


Figure 3

Figure 9 shows the power spectral density (PSD) of transmitted signals. The code applies an FFT to each complete waveform to examine its frequency content.

The upper plot is 4-FSK. Its signal power is concentrated around its four tone frequencies 2550, 2850, 3150 and 3450 Hz

The lower plot is 8-FSK, with tones from 1950 to 4050 Hz. Its power extends across a wider frequency range because it uses eight tones spaced 300 Hz apart.

The PSD plot shows the spectrum, those bandwidth values come from:-

$$B_{approx} = (m-1)\Delta f + \frac{2}{T_S}$$

Aryan Malik 2409S3668

CCE-A Roll no : 58

Channel : AWGN and narrowband interference

Purpose and Model

A telemetry receiver must recover information even when its radio signal is disturbed. I modelled two disturbances for the 4-FSK and 8-FSK waveforms: additive white Gaussian noise (AWGN) which spreads across the sampled frequency band and narrowband interference, represented by one sinusoid at chosen frequency.

The QS(B) channel itself makes no bit decisions so its figures and power measurements do not establish BER or PER.

The input x is clean, real sampled passband waveform waveform produced by QS(A).

$$r[k] = x[k] + n[k] + j(k)$$

$$k = 0, \dots, N-1$$

$r[k]$ is the received sample
N is the number of samples, all three terms have the same waveform amplitude unit.

AWGN generation

The code measures clean average signal power as $P_s = \text{mean}(x[k]^2)$ using normalised 1 Ω reference.

It defines gross bit rate as $R_b = R_s \log_2 M$, where R_s is symbol rate and M is the number of FSK tones. Thus $E_B = P_s/R_b$. This is energy per transmitted bit slot.

for $\left(\frac{E_b}{N_0}\right)_{dB}$ $\gamma_b = 10^{(E_B/N_0)_{dB}}$

$$\sigma_n^2 = \frac{N_0 f_s}{2} = \frac{P_s f_s}{2 R_s \log_2(M) \gamma_b}$$

the factor $\frac{1}{2}$ follows real passband AWGN convention.

Sampling at f_s includes nyquist band of width f_s.

Giving variance $\left(\frac{N_0}{2}\right) f_s$.

Narrowband interference

Jammer is one ~~to~~ continuous sinusoid across the packet

$$j[k] = A \sin\left(2\pi f_j \frac{k}{f_s} + \phi\right).$$

request jammer to signal ratio is

$$JSR_{dB} = 10 \log_{10}(P_j/P_s).$$

The experiment compares a jammer on an allocated tone b/w central tones and one tone spacing above the highest allocated tone.

A between tone has a different frequency relationship to toned detector.

Parameter and Procedure

Parameter	Actual Value
Sampling rate f_s	24000 samples/s
Symbol Rate Rs	300 symbols/s
Samples per symbol	80
Modulation order	4-FSK and 8-FSK.
Tone spacing	300 Hz
4 FSK tones	2550, 2850, 3150, 3450 Hz
8 FSK tones	1950, 2250, 2550, 2850, 3150, 3450, 3750, 4050 Hz
Gross bit rates	600 bits/s for 4FSK, 900 bits/s for 8FSK
E_b/N_0 sweep	0, 2, 4, 6, 8, 10, 12, 14 dB
JSR sweep	−10, 0, +10 dB, plus a no jammer baseline.
Jammer positions	On tone: 2850 Hz between tones: 3000 Hz outside bank: 3750 Hz

nine combinations of 3 JSR values with 3 jammer positions and one no jammer case therefore, makes it $2 \times 8 \times 10 = 160$ cases. Each case is one noise and jammer realisation.

The cases use distinct seeds 3128 + case id's from 3129 to 3288 so the random waveforms can be reproduced without changing MATLAB's global random state.

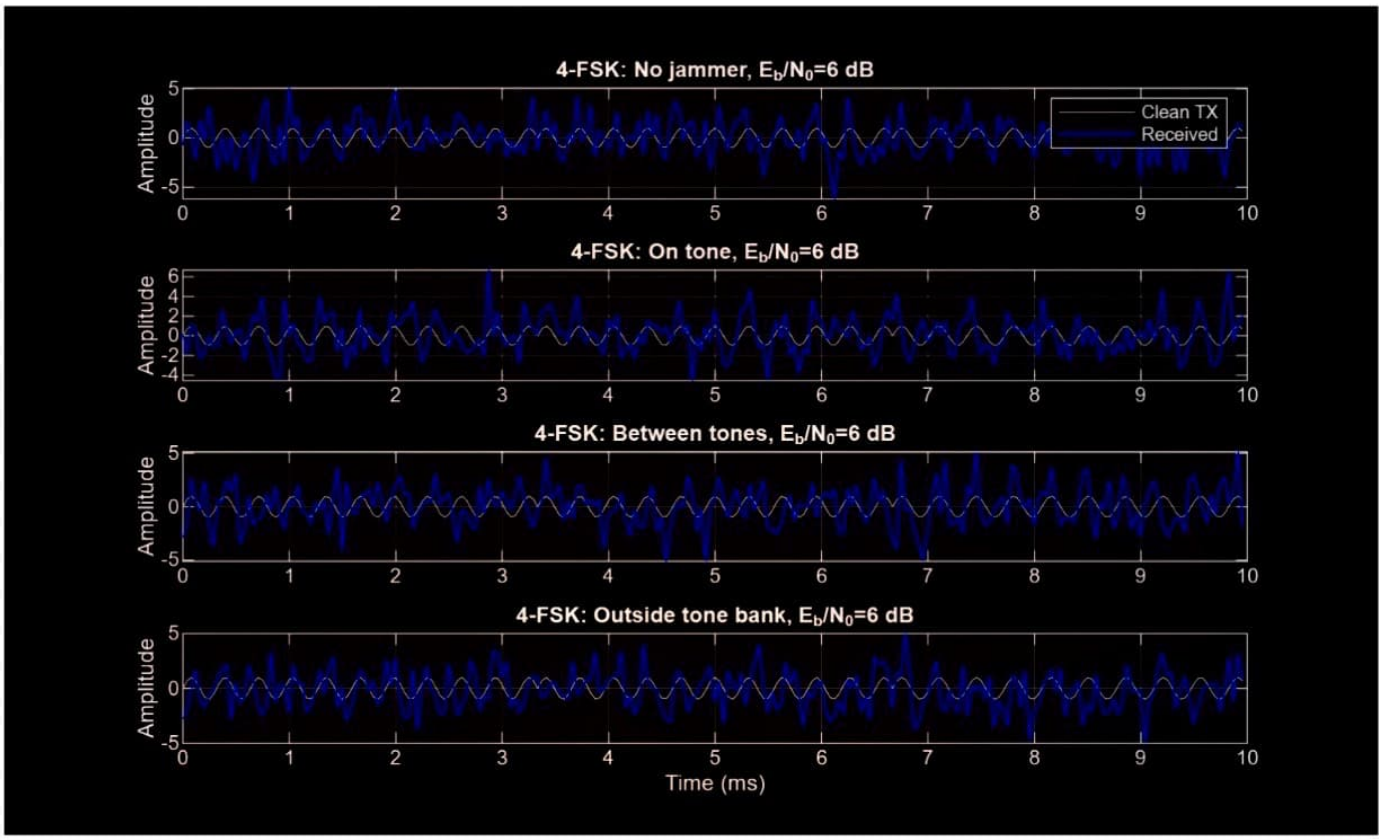
Measured results and observations

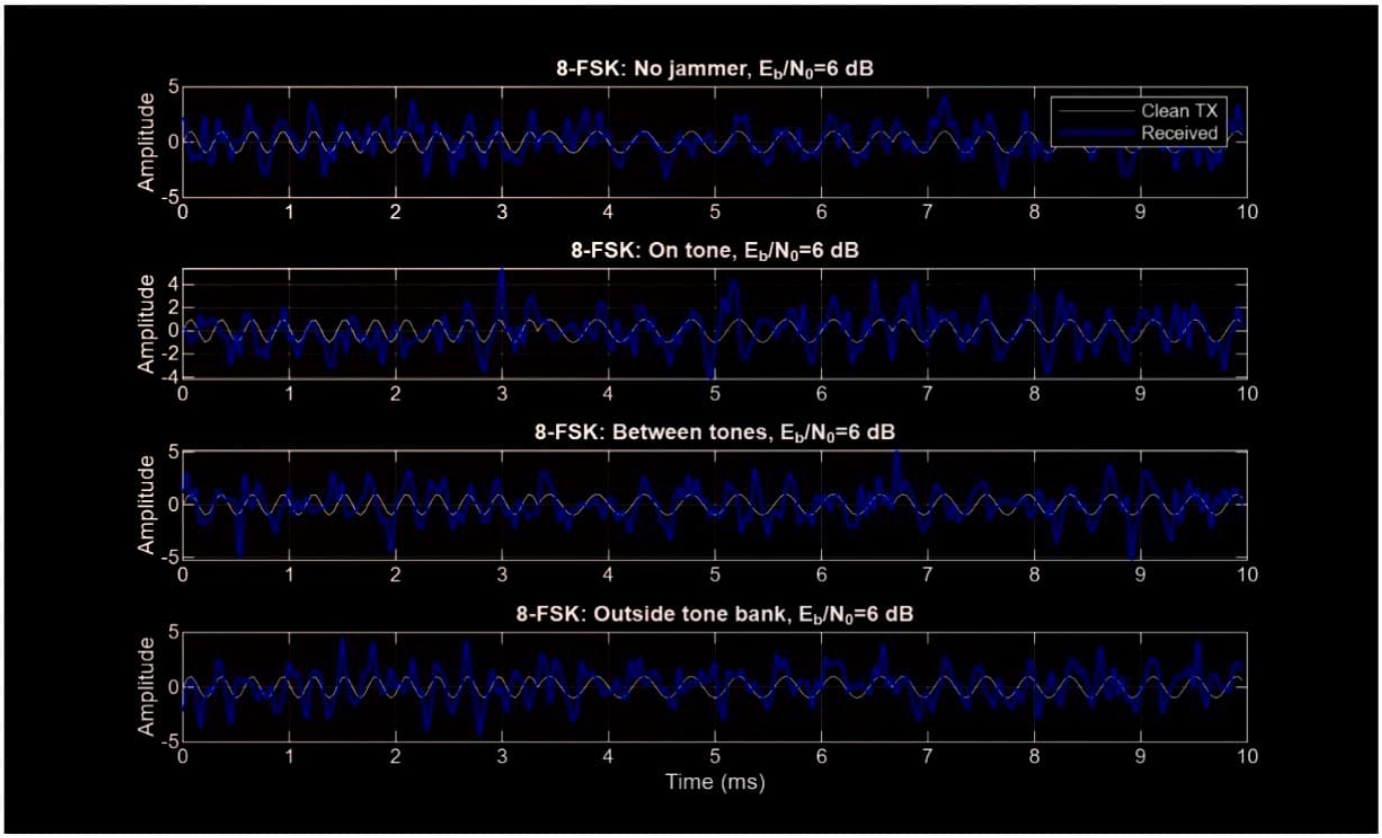
The csv summary has 160 ~~Rows~~ ~~&~~ runs, 80 per modulation order including 16 non jammer case. clean power signal is 0.5 in every case.

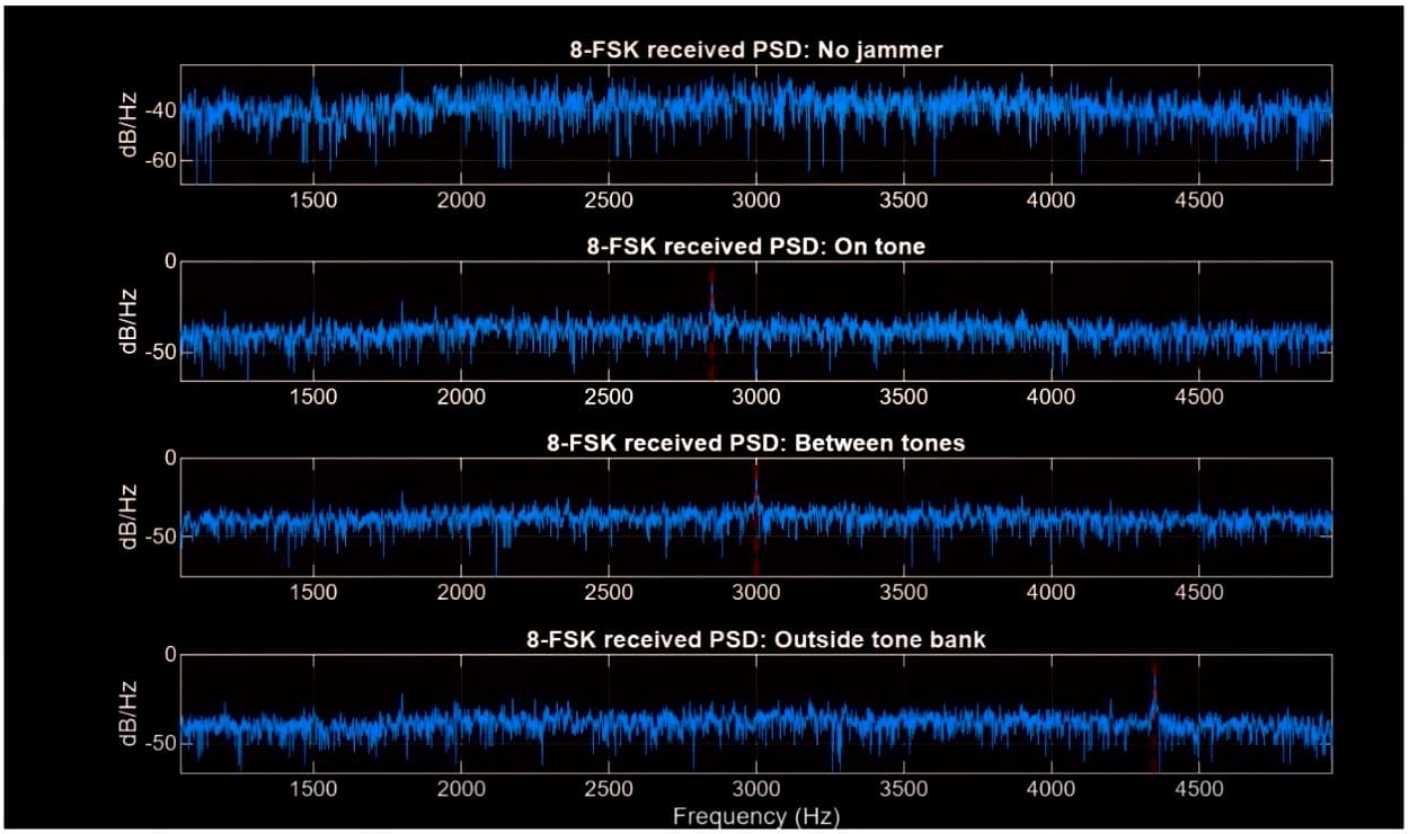
Across all jammer cases, the maximum absolute difference between requested and measured JSR is 2.01×10^{-13} dB. Maximum relative difference between measured and target AWGN power is 3.97 %.

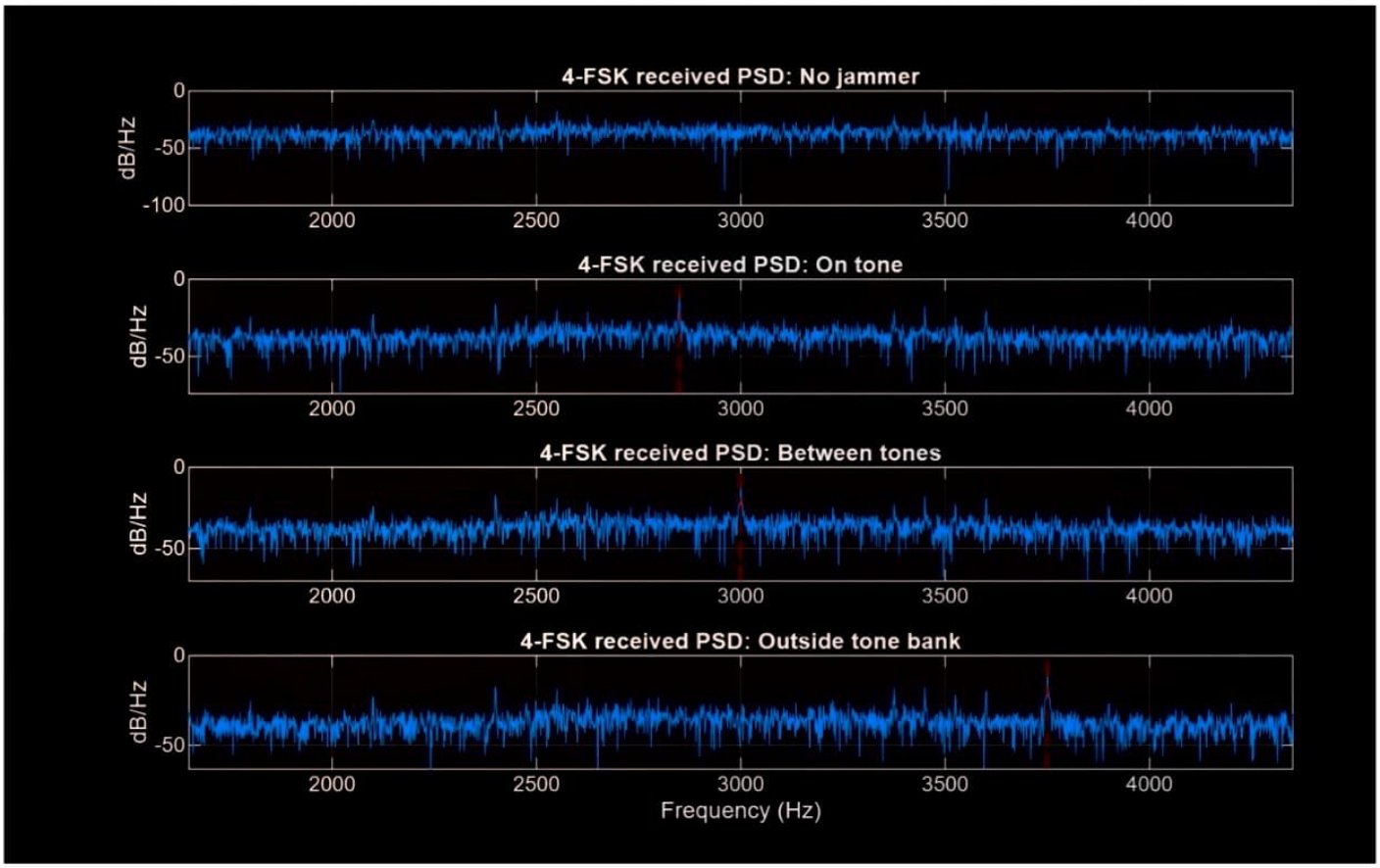
4FSK case 35 uses $\frac{E_b}{N_0}$ = 6dB, JSR = 0 dB and 2850 Hz on tone jammer.

Noise ~~power~~ power is 0.511886 and measured noise power is 0.412941.


4-FSK: No jammer, Eb/N0=6 dB
Clean TX
Received
4-FSK: On tone, Eb/N0=6 dB
4-FSK: Between tones, Eb/N0=6 dB
4-FSK: Outside tone bank, Eb/N0=6 dB
Amplitude
Time (ms)


8-FSK: No jammer, Eb/N0=6 dB
Clean TX
Received
8-FSK: On tone, Eb/N0=6 dB
8-FSK: Between tones, Eb/N0=6 dB
8-FSK: Outside tone bank, Eb/N0=6 dB
Amplitude
Time (ms)


8-FSK received PSD: No jammer
dB/Hz
-40
-60
1500
2000
2500
3000
3500
4000
4500
8-FSK received PSD: On tone
dB/Hz
0
-50
8-FSK received PSD: Between tones
dB/Hz
0
-50
8-FSK received PSD: Outside tone bank
dB/Hz
0
-50
Frequency (Hz)


4-FSK received PSD: No jammer
dB/Hz
0
-50
-100
2000
2500
3000
3500
4000
4-FSK received PSD: On tone
dB/Hz
0
-50
4-FSK received PSD: Between tones
dB/Hz
0
-50
4-FSK received PSD: Outside tone bank
dB/Hz
0
-50
Frequency (Hz)

Q5 (c) RECIEVER DESIGN AND IMPLEMENTATION

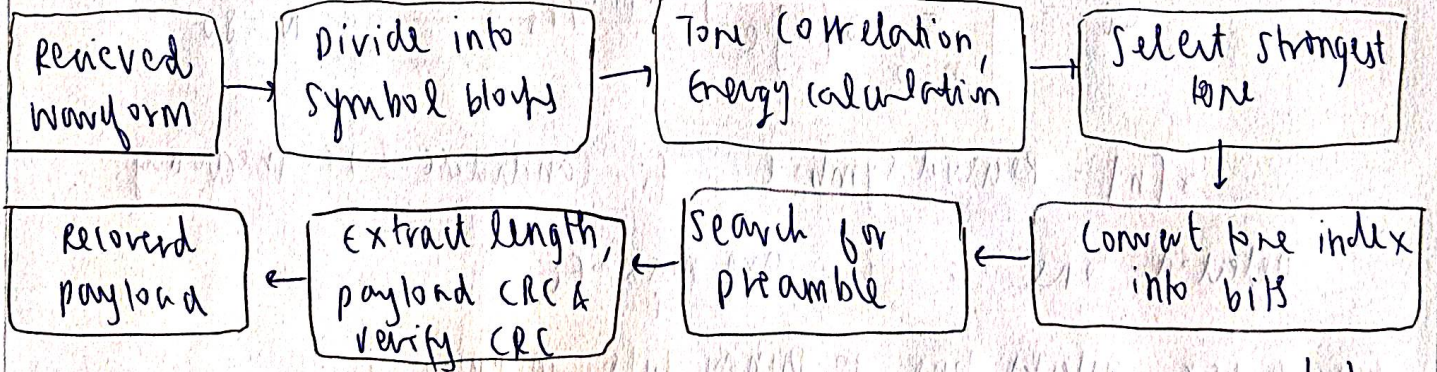
Ayati Rauthan
24095 3138
Ayati

C1. **Aim** : To implement a reciever that recovers registration-number-derived telemetry data transmitted using 4-FSK & 8-FSK through an AWGN channel with narrowband interference.

Reciever Architecture :

The reciever ~~is~~ detects the transmitted tones, converts them into bits, identifies the packet, validates its CRC & reconstructs the orignal payload.

The reciever uses Noncoherent detection, it doesn't require an estimate of the recieved carrier phase. Sampling frequency, symbol boundaries & allocated tone frequencies are assumed to be known. However, the packets starting point is unknown & must be identified using the preamble.



The ~~detected~~ ~~payload~~ bits & packet outcome are also passed to a seperate evaluation stage for BER & PER calculation.

Sampling Freq F_s	24000 samples/s
Symbol Rate, R_s	300 symbols/s
Symbol duration $T_s = \frac{1}{R_s}$	3.33 ms
Samples per symbol $N = \frac{F_s}{R_s}$	80
Modulation orders	M = 4 & M = 8
Bits per symbol $= \log_2 M$	2 & 3
Detection method	Noncoherent correlation energy detection

C2. Noncoherent MFSK DETECTION

The recieved waveform is divided into blocks of 80 samples, with each block representing 1 symbol. For every block, reciever calculates the correlation energy at each allocated tone frequency. For the i^{th} tone, Detection metric:

$$\epsilon_i = \left| \sum_{n=0}^{N-1} r[n] e^{-j2\pi f_i n / F_s} \right|^2 \qquad N = 80$$

$r[n]$ = Recieved symbol f_i - Candidate tone frequency

Detected tone : $\hat{i} = \arg\max_i \epsilon_i$

For 4-FSK, Detected tone to Binary bits mapping is:

Tone frequency	Binary Bits
2950 Hz	00
2850 Hz	01
3150 Hz	10
3450 Hz	11

(For 8-FSK, each tone represents 3 bits instead of 2)

Code 1 : MATLAB implementation of symbol blocking, correlation-energy calculation, strongest tone selection & binary conversion

```
function [bits, diagnostics] = mfsk_detect(rx, detector)
%MFSK_DETECT Detect complete, aligned real-passband symbols; return row bits.
arguments
    rx {mustBeNumeric,mustBeReal,mustBeFinite}
    detector (1,1) struct
end
assert(isvector(rx) && ~isempty(rx), 'q5c:Waveform','rx must be a nonempty vector.');

ns = detector.samplesPerSymbol;
assert(mod(numel(rx),ns)==0,'q5c:IncompleteSymbol','Incomplete symbol block.');
blocks = reshape(double(rx),ns,[]);
scores = abs(blocks.'*detector.reference).^2;
[~, indices] = max(scores,[],2);
k = detector.bitsPerSymbol;
% Power-of-two division gives exact fixed-width mapping without strings.
bitRows = mod(floor((indices-1)./(2.^(k-1:-1:0))),2);
bits = reshape(double(bitRows.'),1,[]);
if nargout>1
    diagnostics = struct('toneIndices',indices,'scores',scores);
end
```

The tone references are calculated once and reused. Matrix operations calculate detection energies for all symbols efficiently.

C3. FRAME SYNCHRONISATION & CRC VALIDATION

After symbol detection, receiver searches the ~~detect~~ decoded bitstream for the packet preamble. The preamble consists of the hexadecimal byte A5 & 5A;

Preamble = 10100101 01011010

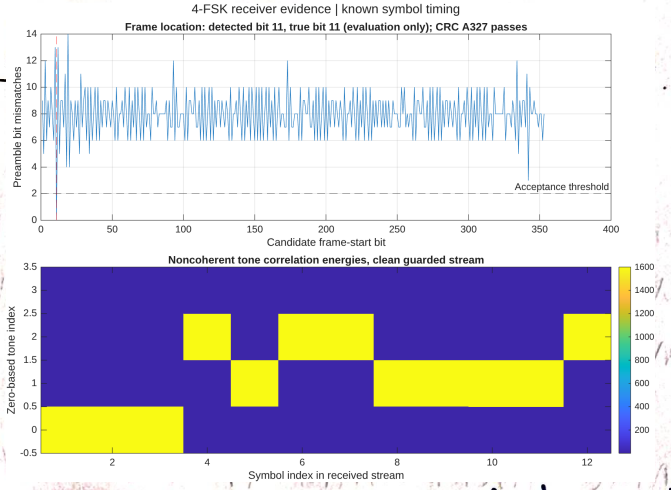
Transmitted frame has the structure :

Preamble	16 bit
Payload-length header	8 bits
Payload	312 bits
CRC	16 bits
Total frame length	352 bits

The length byte contains value 39, indicating 39 payload bytes. 4-FSK waveform requires no padding, 8-FSK waveform contains 2 additional padding bits (excluded from CRC validation). At each possible bit position, receiver counts the mismatches bw the received 16-bit seq & known preamble. Positions

with at most 2 mismatches are considered CANDIDATES. candidates are checked in order of increasing mismatch count. The reciever reads the length byte, checks the complete frame is available & then validates it CRC. If a candidate fails, search continues for another valid frame.

Frame Synchronization



The upper plot shows preamble mismatch count & detected frame position. In this eg, 5 guard symbols preceds the packet, so frame begins at bit 11. Reciever detects this position. The lower plot shows the corelation energies used for Tone selection.

CRC VALIDATION checks if the decoded protected data contains a detectable error (preamble & Modulation-padding bits not included in CRC calculation).

Generator Polynomial = $x^{16} + x^{12} + x^5 + 1$
Polynomial in hexadecimal = 1021
Initial value = FFFF
Input/output reflection = Disabled
Final XOR value = 0000
Protected data - Length byte followed by payload

A candidate is accepted only when these CRC values match.

Independent CRC-16 / CCITT-FALSE calculation using Polynomial-derived lookup table

```
%CRC16_CCITT_FALSE CRC-16: poly 1021, init FFFF, nonreflected, xorout 0000.
arguments
    bytes {mustBeNumeric,mustBeReal,mustBeFinite}
end
assert((isvector(bytes) || isempty(bytes)) && all(bytes(:)>=0 & ...
    bytes(:)<=255 & bytes(:)==fix(bytes(:))), 'q5c:Bytes','Expected byte vector.');
% Immutable lookup cache, derived once from the polynomial (not payload data).
persistent lookup
if isempty(lookup)
    lookup = zeros(1,256,'uint16');
    for n=0:255
        value=bitshift(uint16(n),8);
        for b=1:8
            top=bitget(value,16);
            value=bitshift(value,1);
            if top, value=bitxor(value,uint16(4129)); end
        end
        lookup(n+1)=value;
    end
end
checksum=uint16(65535);
```

Length parsing, payload extraction & CRC verification

```
for start=starts
    if start+23>numel(bits), continue; end
    declared=bits_to_bytes(bits(start+16:start+23));
    last=start+16+8+8*double(declared)+16-1;
    if last>numel(bits), continue; end
    bytes=bits_to_bytes(bits(start+16:last));
    received=bitor(bitshift(uint16(bytes(end-1)),8),uint16(bytes(end)));
    calculated=crc16_ccitt_false(bytes(1:end-2));
    candidate=empty_frame();
    candidate.status='crc_failed';
    candidate.startBit=start;
    candidate.preambleErrors=mismatch(start);
    candidate.declaredLength=double(declared);
    candidate.payloadBytes=bytes(2:end-2);
    candidate.payloadText=char(candidate.payloadBytes);
    candidate.receivedCRC=received;
    candidate.calculatedCRC=calculated;
    candidate.crcOK=(received==calculated);
    candidate.candidateCount=numel(starts);
    if ~haveComplete
        frame=candidate;
        haveComplete=true;
    end
```

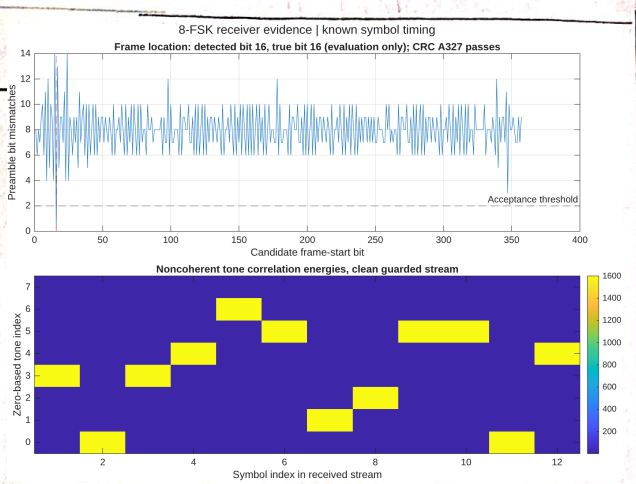
Standard ASCII test req "123456789" produces CRC 29B1, confirming CRC convention. Project's req no payload produces CRC A327.

C4. PAYLOAD RECOVERY & VERIFICATION

Transmitted payload = 240953668 | 240953568 | 240953138 | 240953634
Contain 39 ASCII bytes → ∴ 312 payload bits

Both clean transmitter waveforms decoded successfully. The recovered payload matched the transmitted payload exactly for both 4-FSK & 8-FSK, and calculated CRC matched A327.

Successful Frame Synchronization



When I protected payload bit was deliberately changed, CRC validation failed. This demonstrates receiver rejects corrupted candidate instead of reporting successful recovery.

C5. BER & PER Performance Evaluation

Raw payload BER = $\frac{\text{No. of incorrectly detected payload bits}}{\text{Tot transmitted payload bits}}$

Evaluator uses known transmitted payload position to compare all 312 payload bits in every trial with known evaluation alignment, not supplied to receiver.

End-to-End PER = $\frac{\text{No. of unsuccessful packets}}{\text{Tot transmitted packets}}$

Final Project Settings :

E_b / N_0 = 0 to 14 dB in steps of 2 dB

Baseline - AWGN without a Jammer

Jammer to Signal Ratio - -10, 0 and +10 dB

Jammer placements - On tone, between tone & outside both

Frames per condition = 1000

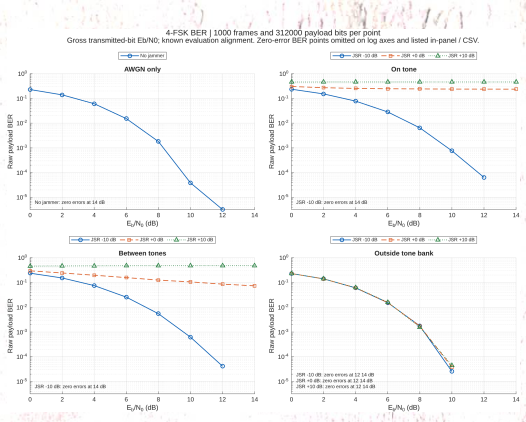
Payload bits per frame = 312

No of conditions = $2 \times 8 \times (1 + 3 \times 3) = 160$

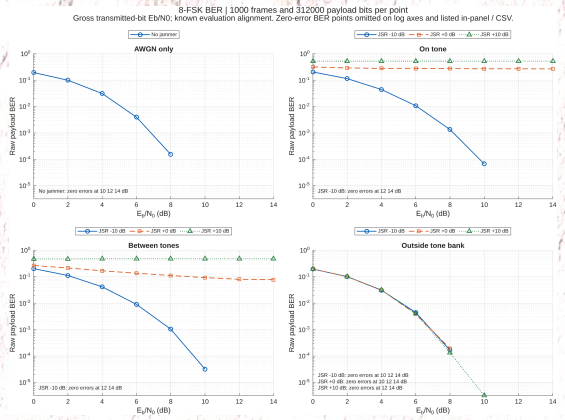
Ayati 240992138

Thus, final project evaluated 160000 frames & 49920000 bits.

BER RESULTS



Raw payload BER for 4-FSK under AWGN & Narrowband interference



Raw Payload BER for 8-FSK under AWGN & Narrowband interference

M	E_b/N_0, dB	Bit errors / Evaluated bits	Raw BER
4	8	565 / 312000	1.81×10^{-3}
4	10	12 / 312000	3.84×10^{-5}
8	8	48 / 312000	1.53×10^{-3}
8	10	0 / 312000	0 observed

For the AWGN baseline, increasing E_b/N_0 reduces the measured BER. A jammer can increase an incoherent tone's detection energy & cause symbol errors.

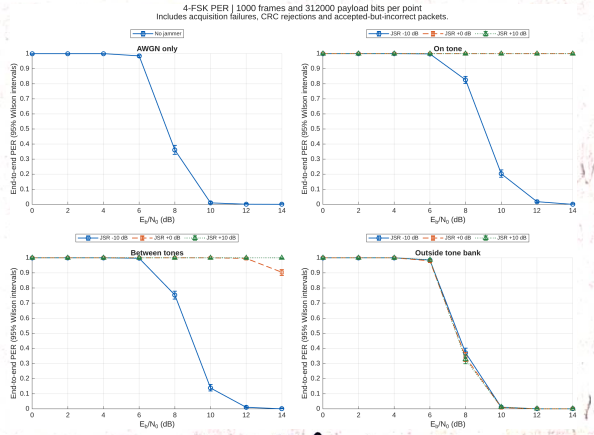
0 observed bit errors do not prove true BER is 0, as it can't be displayed on logarithmic axis

ANALYSIS of 4-FSK BER:

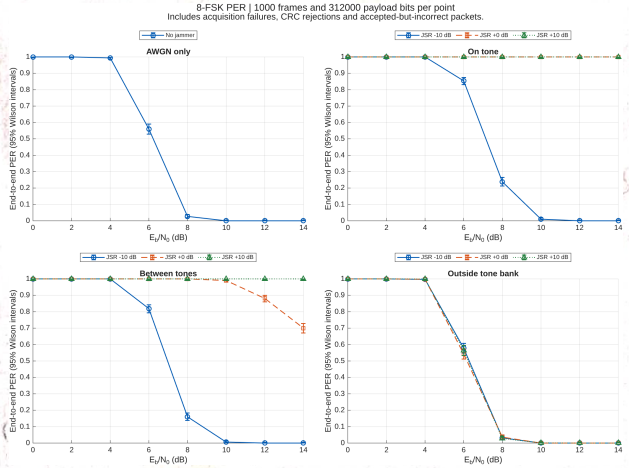
For AWGN alone, BER decreases as E_b/N_0 increases because the received tones become easier to distinguish from noise. At 8 dB, the measured BER is 1.81×10^{-3}, falling to 3.84×10^{-5} at 10 dB. On-tone jammer directly increases energy measured in one detector branch, causing that tone to be selected even when another tone was transmitted.

The between-tone jammer also causes error because a frequency between allocated tones can contribute energy to multiple detector branches over the finite symbol interval. The outside-bank jammer causes much less degradation in this simulation.

PER Results & Conclusion



End-to-end PER for 4-FSK with 2-sided 99% Wilson confidence intervals



8-FSK

Analysis of 4-FSK PER:

For AWGN alone, PER falls from 0.361 at 8 dB to 0.010 at 10 dB, corresponds to reduction from 361 failed packets to 10. PER can remain high even when BER appears relatively small as reception requires correct frame acquisition. At $E_b/N_0 = -14$ dB & JSR = 0 dB, the on-tone PER is 1 & between tone PER is 0.904. Thus, strong interference prevents reliable packet recovery even when thermal noise is low.

Analysis of 8-FSK PER:

For AWGN alone, 8-FSK has PER of 0.027 at 8 dB, corresponding to 27 failures in 1000 packets. At 10 dB, no pkt failures observed.

With JSR = 0 dB & E_b/N_0 = 14 dB, the on-tone PER is 1 & between-tone PER is 0.7. At JSR = +10 dB, both produce PER of 1.

The error bars show two-sided 95% Wilson confidence intervals. For 0 failures in 1000 trials, the upper limit is 0.00383, ∴ experiment doesn't establish perfect reliability.

CONCLUSION

The receiver satisfies required detection, frame synchronization, CRC validation, payload recovery & BER/PER evaluation tasks. Results show improving performance as AWGN decreases while strong in-band interference can cause packet failures.

AKSHAT MANNUR
240953634
GROUP - 5

Q5 (D): MFSK Comparison, Interference Analysis and Engineering Decision

D.1: Objective and Simulation Parameters

The objective of this section is to compare the performance of 4-FSK and 8-FSK modulation schemes for a tdemchy receiver operating under AWGN and narrowband interference. The comparison is based on bandwidth requirements, Bit Error Rate (BER), Packet Error Rate (PER) and sensitivity to jammer frequency and power.

The simulation uses a sampling frequency of 24000 Hz, a symbol rate of 300 symbol/s and a tone spacing of 300Hz. Therefore, the symbol duration is $T_s = \frac{1}{R_s} = \frac{1}{300} = 3.333$ ms

The tone spacing satisfies the orthogonality condition $\Delta f = \frac{1}{T_s}$ for the complex-baseband model used for MFSK.

the experiment evaluates 1000 frames and 312,000 payload bits per condition. The jammer to signal ratio (JSR) is varied across −10 dB, 0 dB, +10dB, in addition to an AWGN-only baseline. Three jammer placements are considered: on-tone, between-tones and outside tones.

D.2 Comparison of 4-FSK and 8-FSK

In M-ary FSK, each frequency (symbol) represents $\log_2 M$ bits. Hence, 4 FSK transmits 2 bits/sym whereas 8-FSK transmits 3 bits/sym.

The gross bit rate is calculated using $R_b = R_s \log_2 M$

For 4-FSK = $R_b = 300 \times 2 = 600$ bit/s

For 8-FSK = $R_b = 300 \times 3 = 900$ bit/s

Thus, 8-FSK provides a 50% higher gross bit rate at the same symbol rate. However, it requires more frequency tones and a wider frequency allocation. The 4-FSK tone frequencies are 2550, 2850, 3150 and 3450 Hz. Their tone-center span is B_4 = 3450 - 2550 = 900 Hz

The 8-FSK frequencies range from 1950 Hz to 4050 Hz giving.

B_8 = 4050 - 1950 = 2100 Hz

These values represent the tone-center span not the exact occupied bandwidths:

Parameter	4FSK	8FSK
Bits / symbol	2	3
Symbol Rate	300 sym/s	300 sym/s
Gross bit Rate	600 bit/s	900 bit/s
Tone Center span	900 Hz	2100 Hz
Relative Bandwidth requirement	Lower	Higher.

8-FSK is preferable when higher throughput is required and sufficient bandwidth is available.

Conversely, 4FSK is more suitable when frequency allocation is limited.

D.3 BER and PER Performance under Noise and Interference

BER and PER provide complimentary measures of communication reliability. Under AWGN conditions only, the BER generally decreases as E_b/N_0 increases because the signal becomes stronger relative to noise.

The BER plots show improved performance for 8-PSK over the displayed sweep. However, the final selection must also account for packet reliability and bandwidth. Narrowband interference causes additional degradation depending on the jammer's frequency and power. As JSR increases, the jammer becomes stronger relative to transmitted signal, making correct tone detection more difficult.

At $\frac{E_b}{N_0}$ = 10 dB and JSR = 0dB, the measured results are:

Configuration	BER	PER
4 FSK, on-tone jammer	0.2391	1.000
8-FSK, on-tone jammer	0.2693	1.000
4-FSK, between-tones jammer	0.1048	1.000
8-FSK, between-tones jammer	0.1214	0.990
4-FSK, outside band jammer	0.000013	0.009
8-FSK, outside band jammer	0	0

one tone jammer produces severe degradation because its frequency ops a valid transmission tone. At the specified operating point, both configuration ce 100% packet failure in the 1000-tested frames.

ween tones jammer also produces some interference. Although its frequency centered on a valid tone, its energy can affect the receiver's tone-energy ements and lead to incorrect symbol decisions.

The outside-band jammer produces substantially less degradation in these tested conditions because it lies away from the valid transmission tones.

D.4 : Identification of the Worst Jammer Placement.

The simulation shows that the effect of narrowband interference depends on the jammer's frequency, its power relative to the signal, and the modulation configuration.

At E_b/N_0 = 10 dB and JSR = 0dB, the on-tone jammer is particularly destructive. It directly overlaps a valid tone, and can be interpreted as the transmitted symbol by the receiver. The resulting incorrect decisions increase BER and can cause packet rejection through CRC validation. The between tones jammer also produces significant degradation. At higher JSR its energy can interfere with the receiver's tone measurements sufficiently to cause packet-failures. At JSR = +10 dB, the measured packet failure rate is 100% for both on-tone and between-tones interference in both configurations.

Therefore, the on-tone and between-tones placements are the most critical cases in the tested conditions. Their relative severity depends on the modulation order, operating point and performance metric, so neither should be declared universally worst without specifying these conditions.

The outside-band placement performs considerably better at the stated $\frac{E_b}{N_0}$ = 10dB and JSR = 0dB operating point

D.5 Final Engineering Decision and Conclusion.

The final configuration should be selected according to the telemetry system's bandwidth and reliability requirements.

If bandwidth is the primary constraint and interference is expected to be limited 4-FSK is a reasonable choice because its center-tone span is 900 Hz, compared to 2100 Hz for 8-FSK. If higher throughput is required, and a wider frequency allocation is acceptable, 8-FSK provides a GBR of 900 bit/s compared to 600 bit/s for 4-FSK.

However, the simulation demonstrates that neither configuration can maintain packet delivery under all tested jammer conditions. At JSR = 0dB, the on-tone jammer causes all 1000 tested packets to fail both modulation orders.
At JSR = 10dB, both on-tone and between tone interference cause complete packet failure in all tested trials.
If the telemetry link must operate reliably in the presence of strong narrowband interference, neither modulation scheme alone is sufficient.

The results are based on finite simulation trials and selected jammer frequencies. Zero observed errors do not prove that the true error probability is zero. For example, observing zero packet failures in 1000 trials gives an approximate 95% upper confidence bound of 0.3% using the rule of three.

In conclusion, 4-FSK offers bandwidth advantage, while 8-FSK offers higher gross throughput. The final choice must be based on explicit bandwidth and reliability targets rather than throughput alone.

Simulation Results and Figures

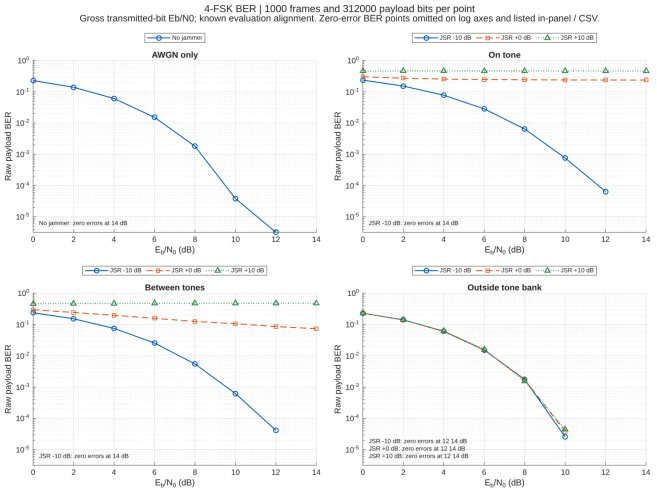


Figure 1. BER vs Eb/N0 — 4-FSK

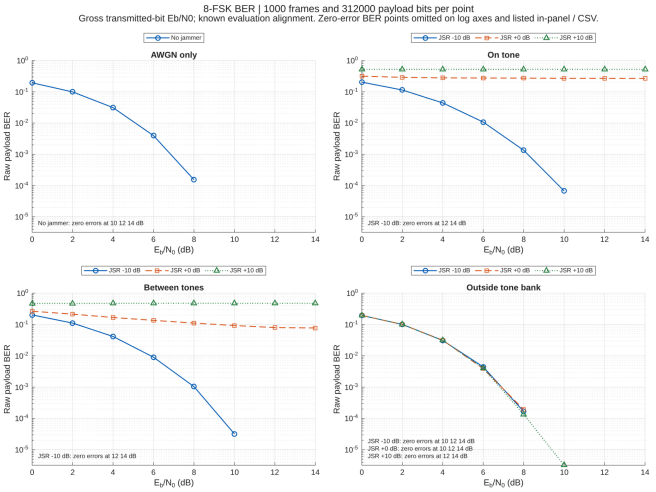


Figure 2. BER vs Eb/N0 — 8-FSK

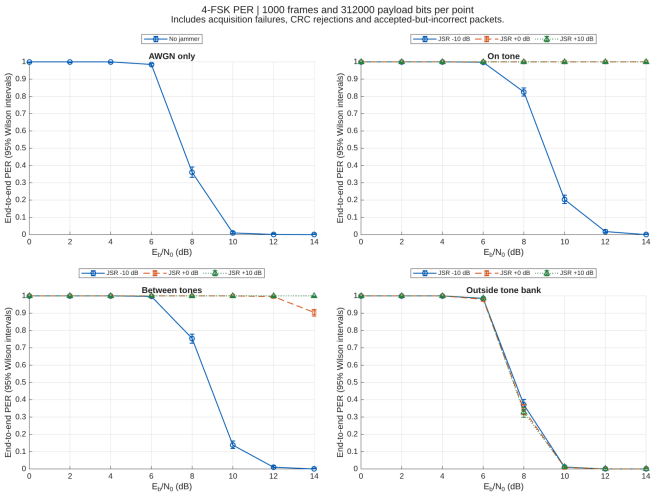


Figure 3. PER vs Eb/N0 — 4-FSK

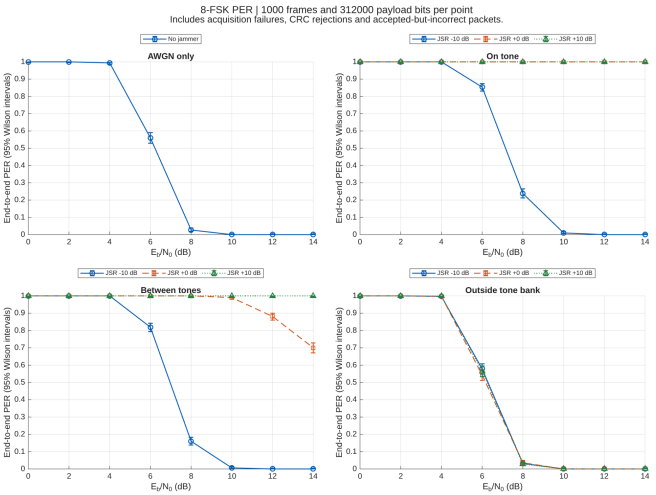


Figure 4. PER vs Eb/N0 — 8-FSK

Simulation Results and Figures

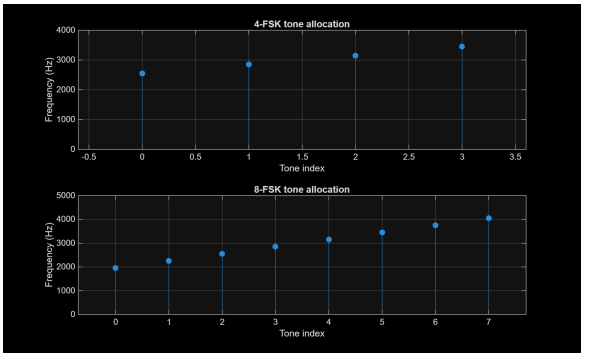


Figure 5. MFSK Tone Allocation

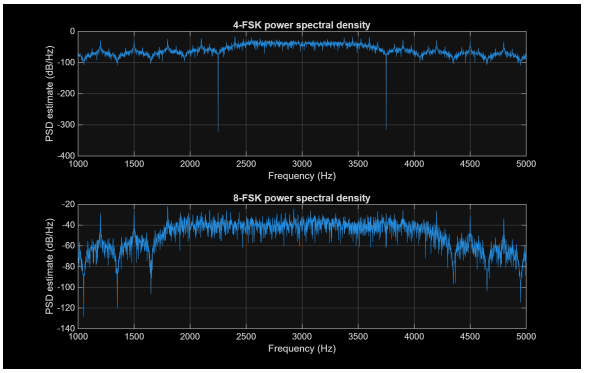


Figure 6. Transmit Power Spectral Density

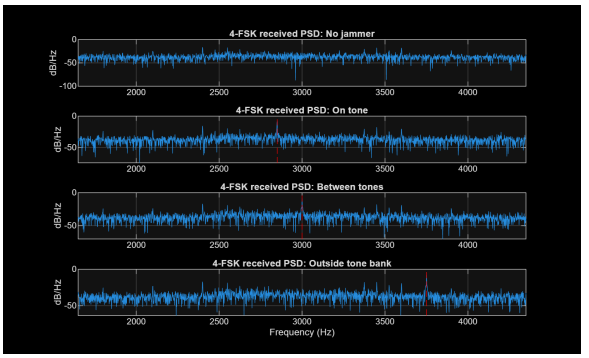


Figure 7. Received Spectrum — 4-FSK

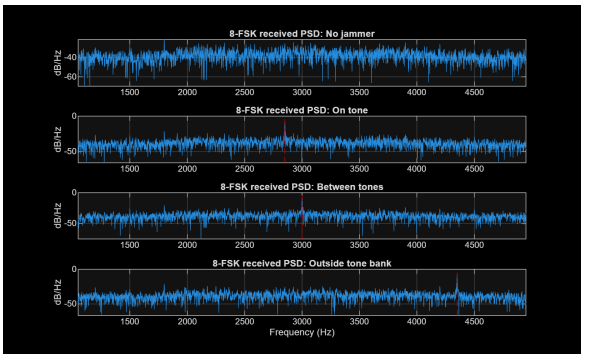


Figure 8. Received Spectrum — 8-FSK